

# Verifikation verteilter Systeme in Isabelle

## Ein Vergleich von Kleppmann und I/O-Automaten

Sepp Stage

TU Chemnitz, Fakultät Betriebssysteme



TECHNISCHE UNIVERSITÄT  
CHEMNITZ

## 1. Einleitung

## 2. Das Zwei-Phase-Commit-Protokoll

- Übergänge des Koordinators

- Übergänge eines Teilnehmers

- Beispielhafte Abläufe

## 3. I/O Automaten

- Signatur

- Mathematische Definition

- Ausführung

- Komposition

## 4. Vergleich

- Modellierung

- Invarianten

- Beweis einer Invarianten

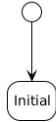
- Beweis der einheitlichen Zustimmung

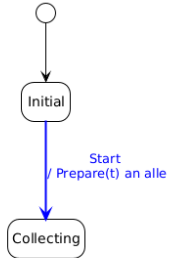
## 5. Vergleich

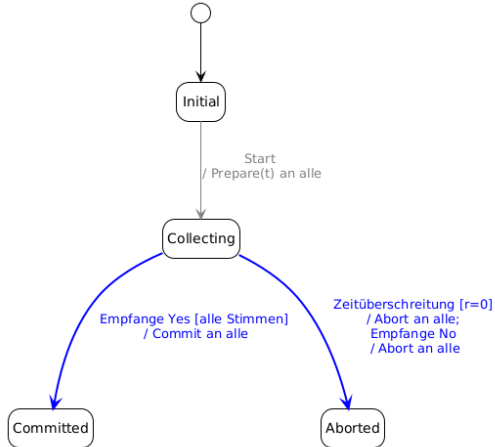
- Kriterien...

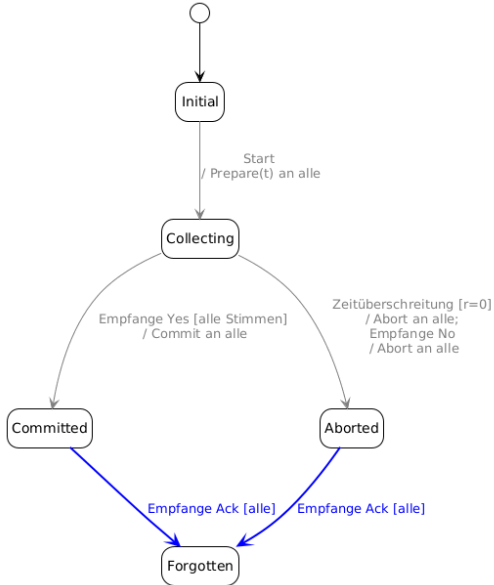
## 6. Fazit

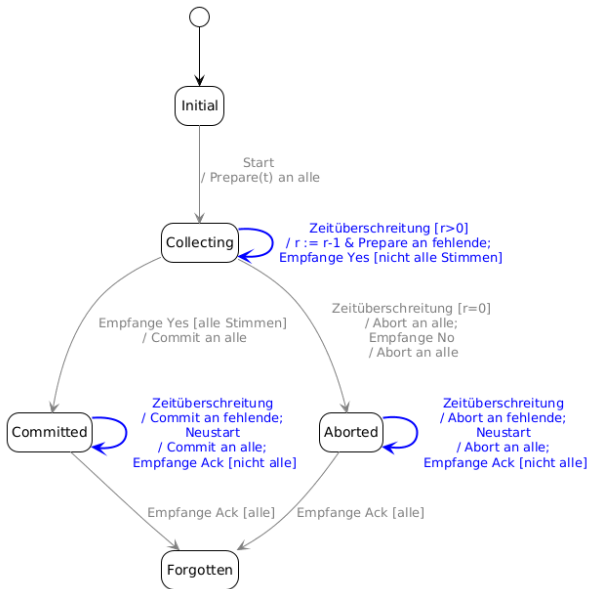
# AT&T



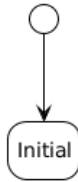


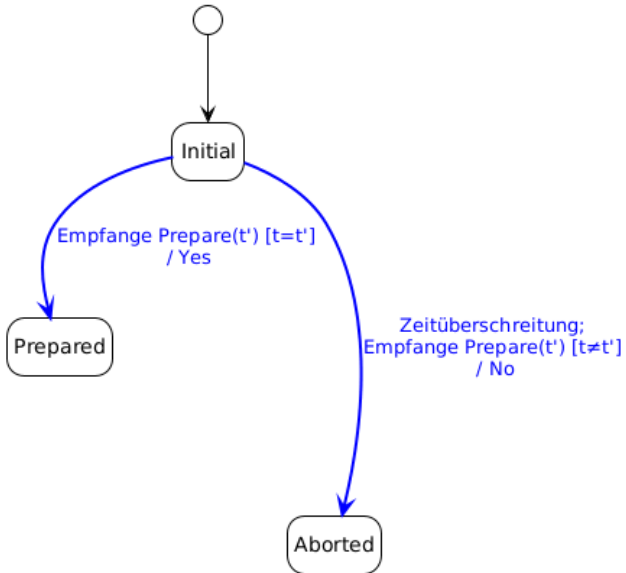


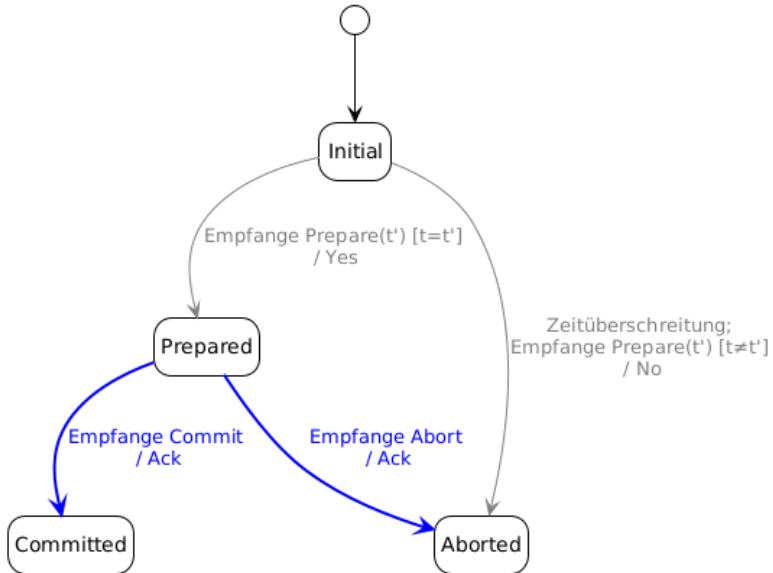


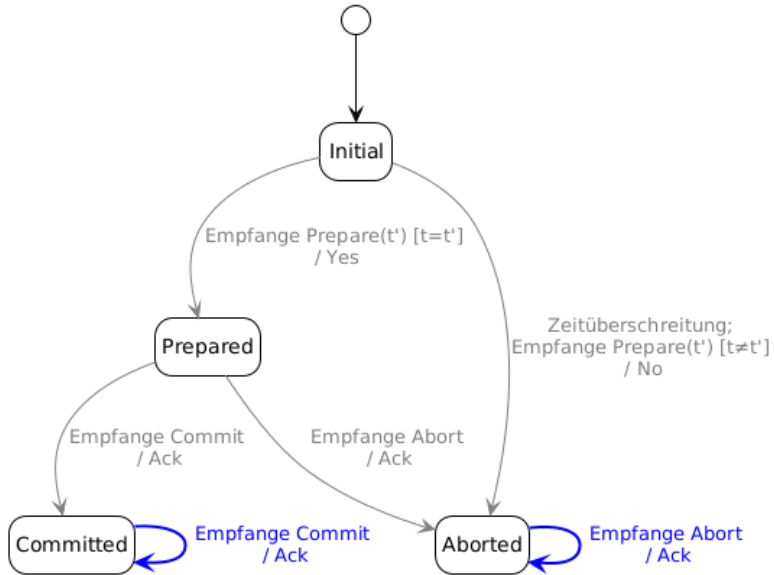


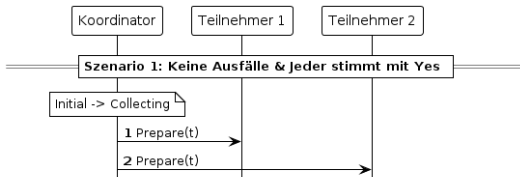


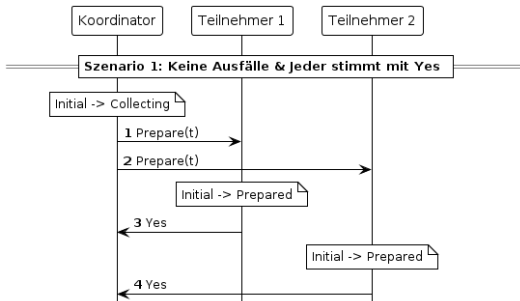


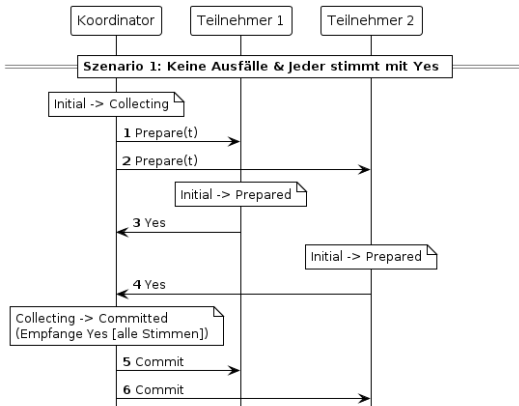


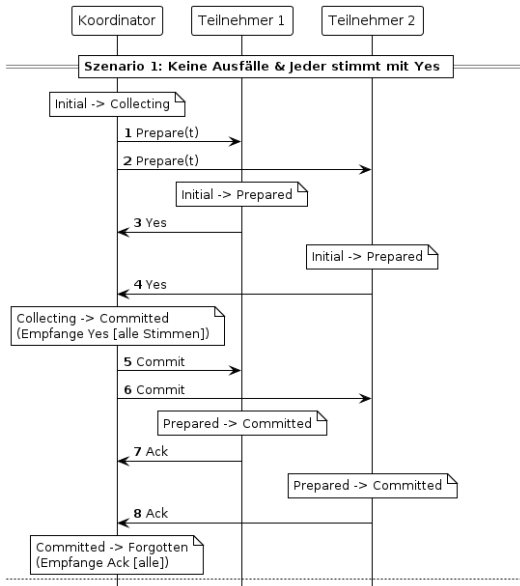




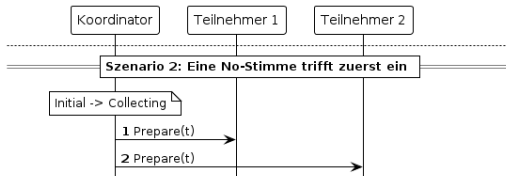


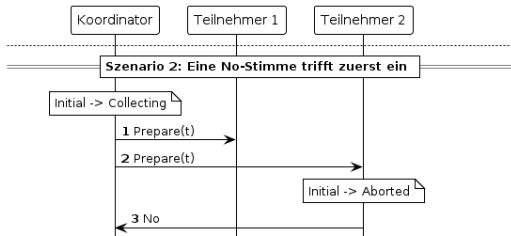


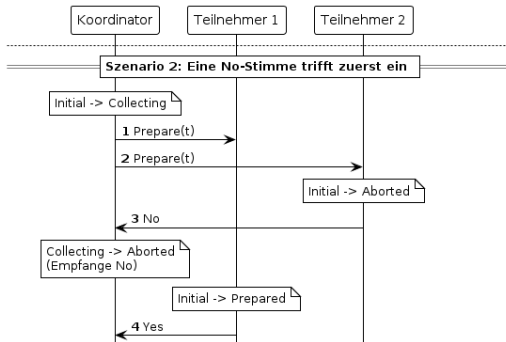


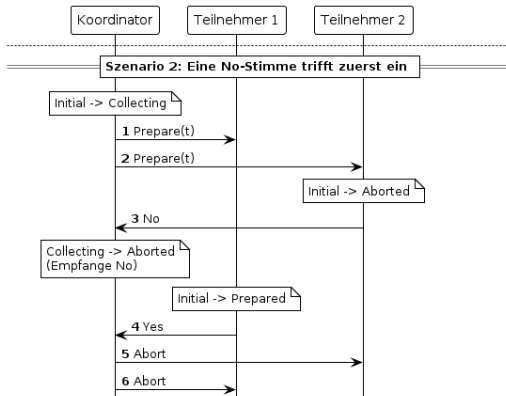


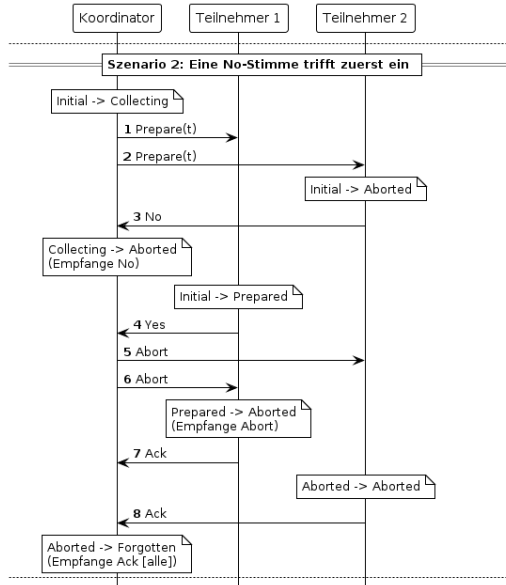


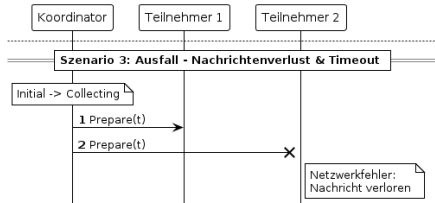


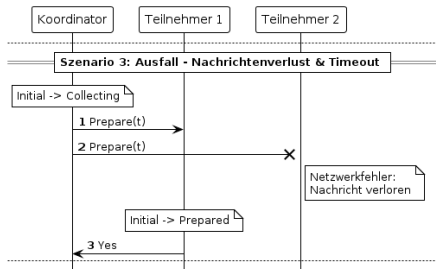


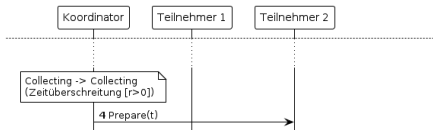
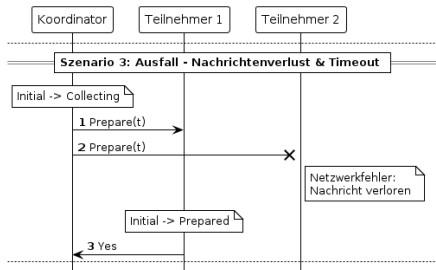




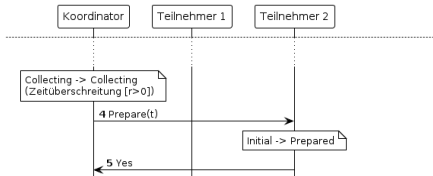
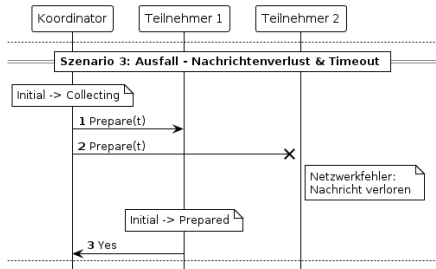


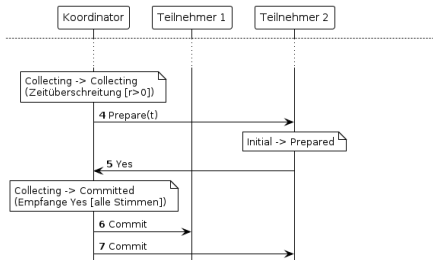
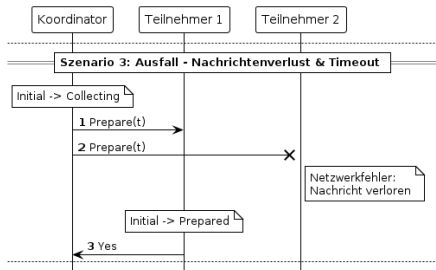


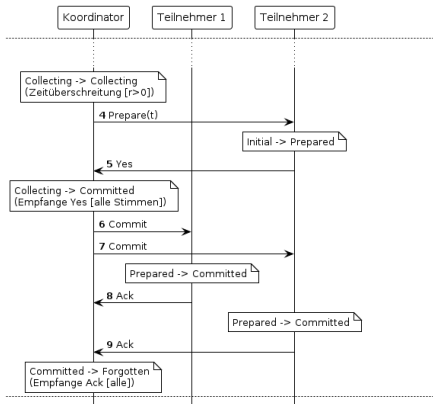
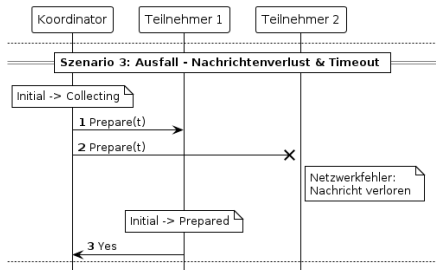


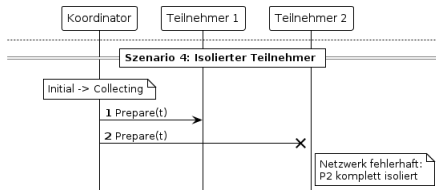


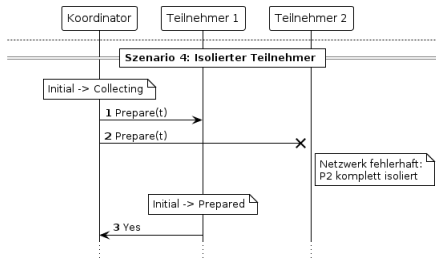


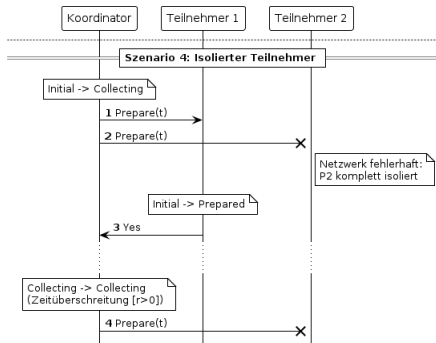


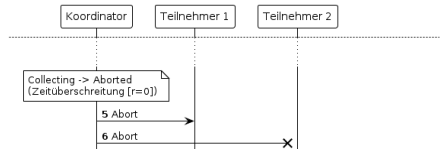
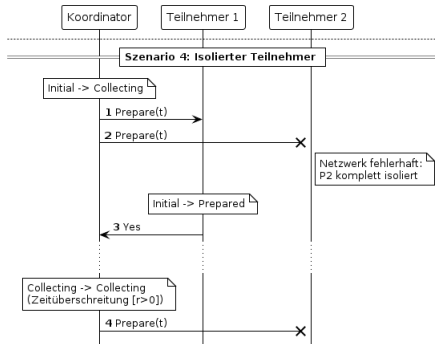


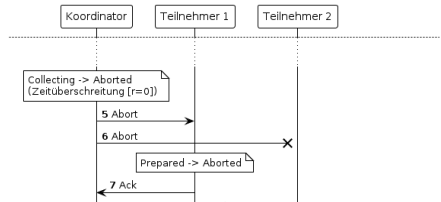
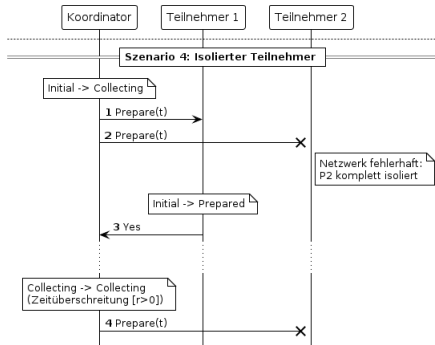




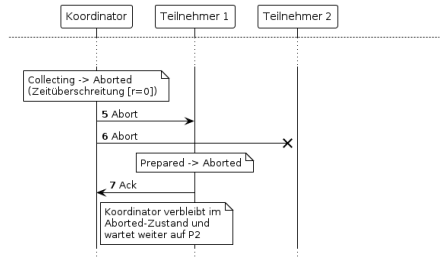
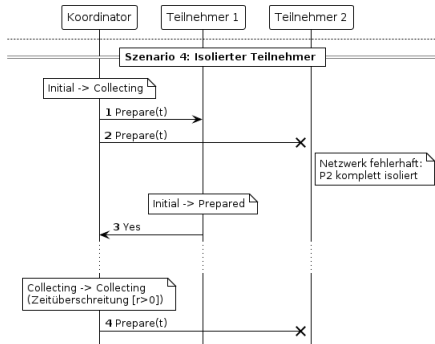


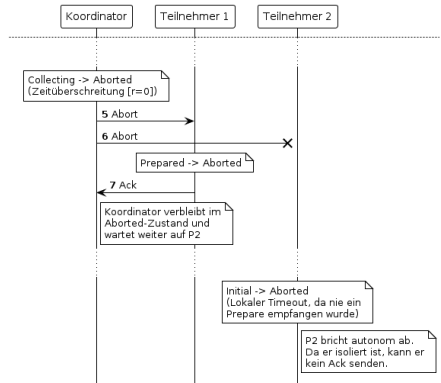
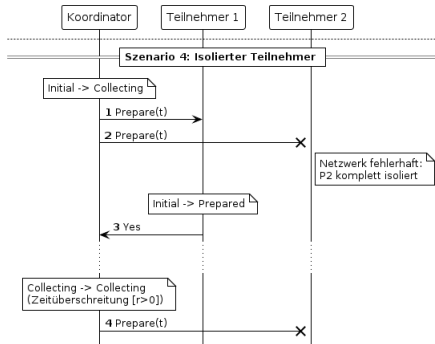












## Definition einer Signatur $S$

- ▶ *Input-Übergänge*  $in(S)$
- ▶ *Output-Übergänge*  $out(S)$
- ▶ *Internal-Übergänge*  $int(S)$

Beispielbild!!!!

## Definition eines I/O Automaten $A$

- ▶ *Signatur*  $sig(A)$
- ▶ *Menge an Zuständen*  $states(A)$
- ▶ *nichtleere Menge an Startzuständen*  $start(A) \subseteq states(A)$
- ▶ *Übergangsrelation*

$$trans(A) \subseteq states(A) \times acts(sig(A)) \times states(A)$$

- ▶ *Aufgabenpartition*  $tasks(A)$

Beispielbild????

## Ausführung eines I/O Automaten $A$

- ▶ *Ausführungsfragment*  $s_0, \pi_1, s_1, \pi_2, \dots$
- ▶  $\forall i \geq 0. (s_i, \pi_{i+1}, s_{i+1}) \in \text{trans}(A)$
- ▶ *Ausführung falls*  $s_0 \in \text{starts}(A)$

Beispielbild????

## Komposition zweier I/O Automaten $A$ & $B$

- ▶ *Automaten müssen kompatibel sein*
- ▶ *Signatures werden Verknüpft*
- ▶ *entstehende Zustände sind Vektoren  $(s_a, s_b)$  mit  $s_i \in \text{start}(I)$*
- ▶  *$A$  und  $B$  dürfen bei gemeinsamen Übergängen gleichzeitig agieren*

OUTPUTS bleiben OUTPUTS für weitere Komposition!

```

10 datatype ('proc, 'msg) event
11     = Start
12     | Receive (msg_sender: 'proc) (recv_msg: 'msg)
13     | Timeout
14     | Restart
  
```

---

```

8  datatype ('proc, 'msg) action =
9      Start "'proc"
10     | Send "'proc" "'proc" "'msg"
11     | Receive "'proc" "'proc" "'msg"
12     | Timeout "'proc"
13     | Restart "'proc"
  
```

```

50 definition coord_asig :: "'proc  $\Rightarrow$  ('proc,'t) action
     $\hookrightarrow$  signature" where
51 "coord_asig pid =
52   ({Receive q pid m | q m. True}  $\cup$  {Start pid, Timeout pid,
     $\hookrightarrow$  Restart pid},
53   {Send pid q m | q m. True},
54   {})"

89 definition parts_asig :: "'proc set  $\Rightarrow$  ('proc,'t msg) action
     $\hookrightarrow$  signature" where
90 "parts_asig P =
91   ({Receive q p m | q p m. p  $\in$  P}  $\cup$  {Timeout p | p. p  $\in$  P}  $\cup$ 
     $\hookrightarrow$  {Restart p | p. p  $\in$  P},
92   {Send p q m | p q m. p  $\in$  P},
93   {})"

113 definition chan_asig :: "('proc,'t) action signature" where
114 "chan_asig =
115   ({Send s r m | s r m. True},
116   {Receive s r m | s r m. True},
117   {})"
  
```





```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:""'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

---

```

61 datatype 't pstate = PInitial (transaction: 't)
62 | Prepared
63 | PCommitted
64 | PAborted
65 datatype ('t, 'proc) part_state = PState "'t pstate" "('proc
   ↪  × 'proc × 't msg) set"
  
```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

106 type_synonym ('proc, 't) chan_state = "('proc × 'proc × 't
   ↪  msg) set"
  
```

"By itself, the I/O automaton model has very little structure, which allows it to be used for modelling many different types of distributed systems. Additional structure must be added to the basic model to enable it to describe particular types of asynchronous systems." Lynch buch s.199

blblblb